Analyzing Algorithms to Solve the 2-D Closest Pair Problem

By: Lydia Ballard and Sydney Frisbee

Course: CSC2400-003

Instructor: Dr. Christina Radian

Date: April 20, 2026

Overview

This project explores methods for solving the closest pair problem, which involves finding the two closest points in a metric space, typically using Euclidean distance, among a set of *n* points. To address this problem, four algorithms are analyzed: Brute Force, Divide and Conquer, Line Sweep, and Bee Colony Optimization.

Objective/Research Questions

This study addresses two primary questions: how the runtime of each algorithm scales with increasing input size *n*, and at what input size Divide & Conquer outperforms Brute Force. These questions were chosen for their direct relevance to the central problem. Together, they help identify which algorithm most efficiently solves the closest pair problem.

Algorithms Used

Brute force figures out the distance between every pair and stores the minimum distance found. This algorithm is very straightforward, but more time and resource inefficient because it checks every possible pair of points. It has a time complexity of Θ(n^2), and it is our baseline for comparison to see how much other algorithms can improve against it.

Divide and Conquer is a core/fundamental algorithm for how the closest pair problem works. In the Divide-and-Conquer algorithm, the points are recursively divided into smaller groups. The minimum distance is calculated within each group, and during the merging step, points are checked for possible closer pairs across the dividing point. With a time complexity: Θ (n log n), the algorithm can reduce the time complexity compared to just brute force, so it's better for more data; it is more complex to understand and implement

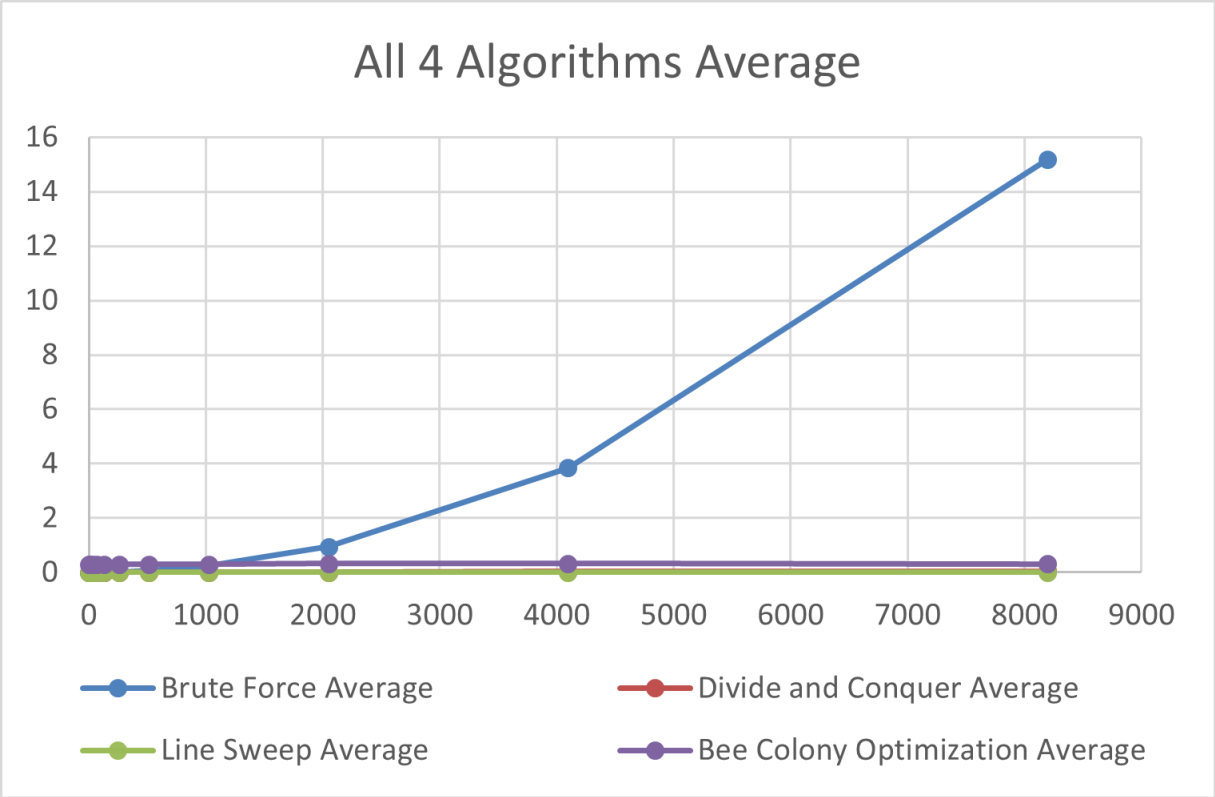
The Line Sweep algorithm is a more specialized algorithm, and it can be useful to compare against more general algorithms. Initially, all points are sorted based on their x-coordinates. This allows the "sweep line" to move from left to right, processing points in an ordered manner. Starts iterating over all the points in increasing order of their x-coordinates. With its Θ (n log n) time complexity, its primary strength is efficiency. However, this comes at the cost of increased implementation complexity.

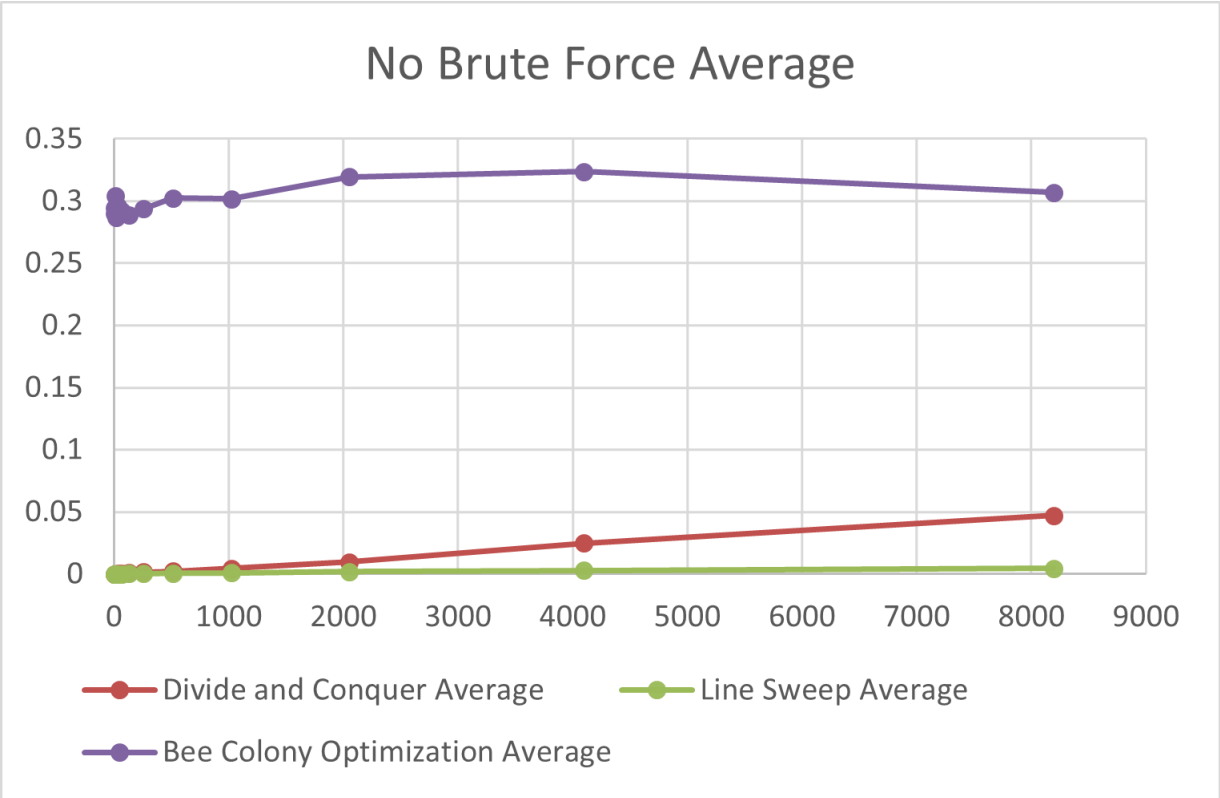
Bee Colony Optimization (BCO) is our nature-based algorithm. This algorithm has 3 different stages/types of bees: Employed Bees, Onlooker Bees, and Scout Bees. Employed Bees are associated with specific food sources, and they explore the neighborhood of their assigned source to find better, closer food. Onlooker Bees wait in the hive and choose a food source to exploit based on the information shared by employed bees through a waggle dance. If a solution cannot be improved over a set number of cycles ("limit"), the employed bee abandons it and becomes a Scout Bee, which searches for a new random pair of food sources. When translating this to code, the Employed Bee section took the current pair and compared it to a neighboring pair, keeping the closer one. The Onlooker Bee Section computed fitness for all pairs and randomly picked a promising pair to try and find a closer distance. Finally, the Scout Bee section determined that if the pair hasn't improved after a specific number of iterations, then it may be a bad candidate and replaces the pair with a random pair. It has a time complexity of Θ (N·D·I), where N is the number of bees (population size), D is the dimensionality of the problem (number of variables), and I is the number of iterations (cycles). BCO works in complex spaces and is good for approximate solutions, however this means it's not exact, and it is slower than specialized algorithms like Divide and Conquer. Because of this more heuristic approach, it can be a good comparison against more exact and widely used algorithms.

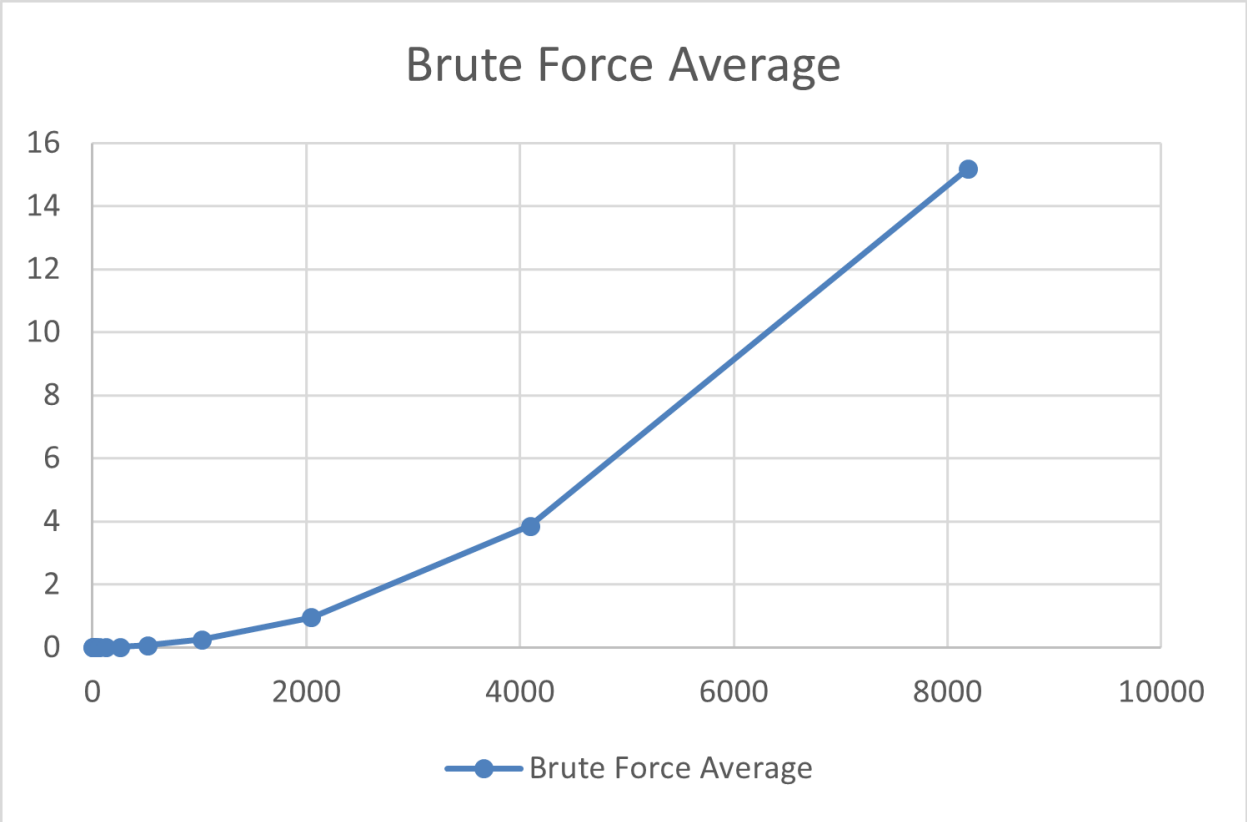
Description of Experiment

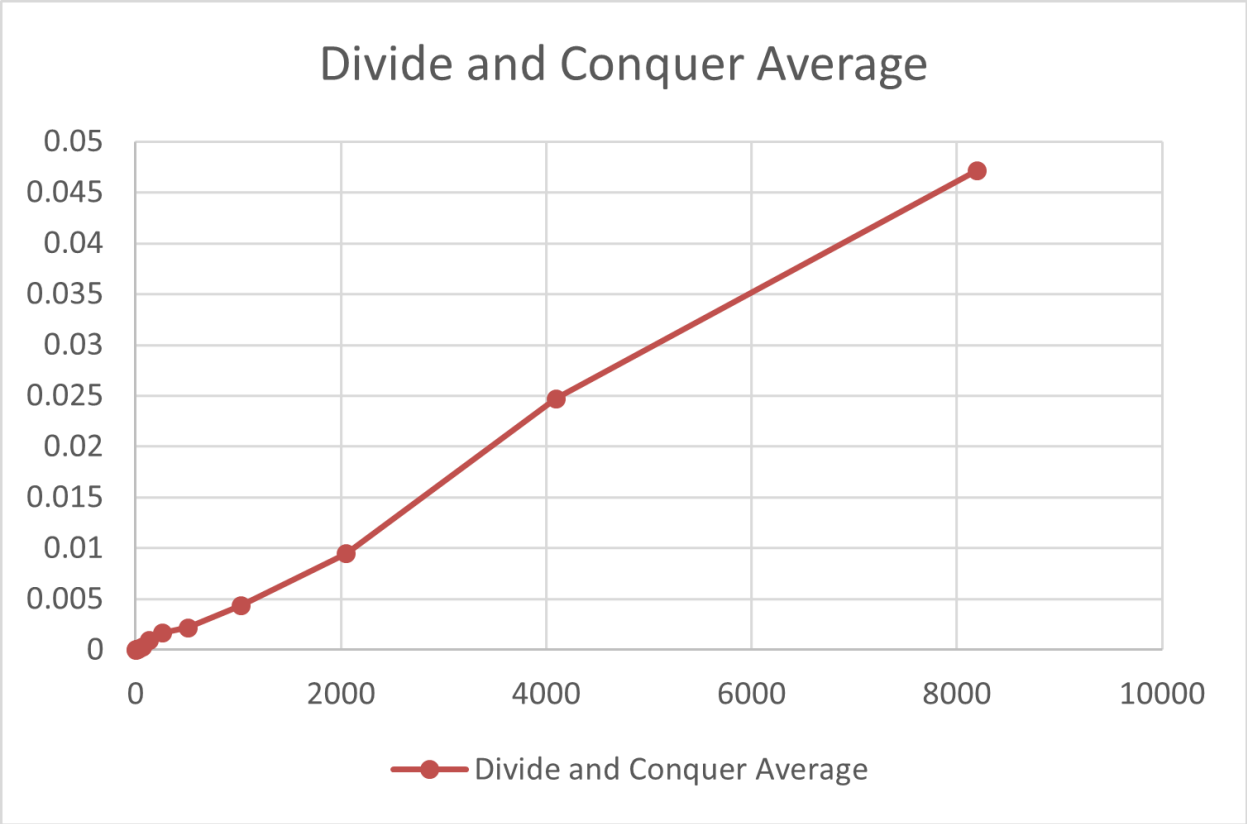
We wrote our algorithms in Python, which each algorithm having its own file that is run through a main file. Datasets were artificially generated of XY pairs with a size n, which starts at 2 and increases exponentially up to 8192. Each algorithm is run 5 times within the program for each input size n to get a more accurate result. Our reference algorithm was Brute Force, and we measured the amount of time each algorithm took to execute in seconds. The machine we conducted our experiment on had an AMD Ryzen 9 6900HS Creator Edition CPU, a Nvidia GeForce 3050 Laptop GPU, and 32 GB of RAM.

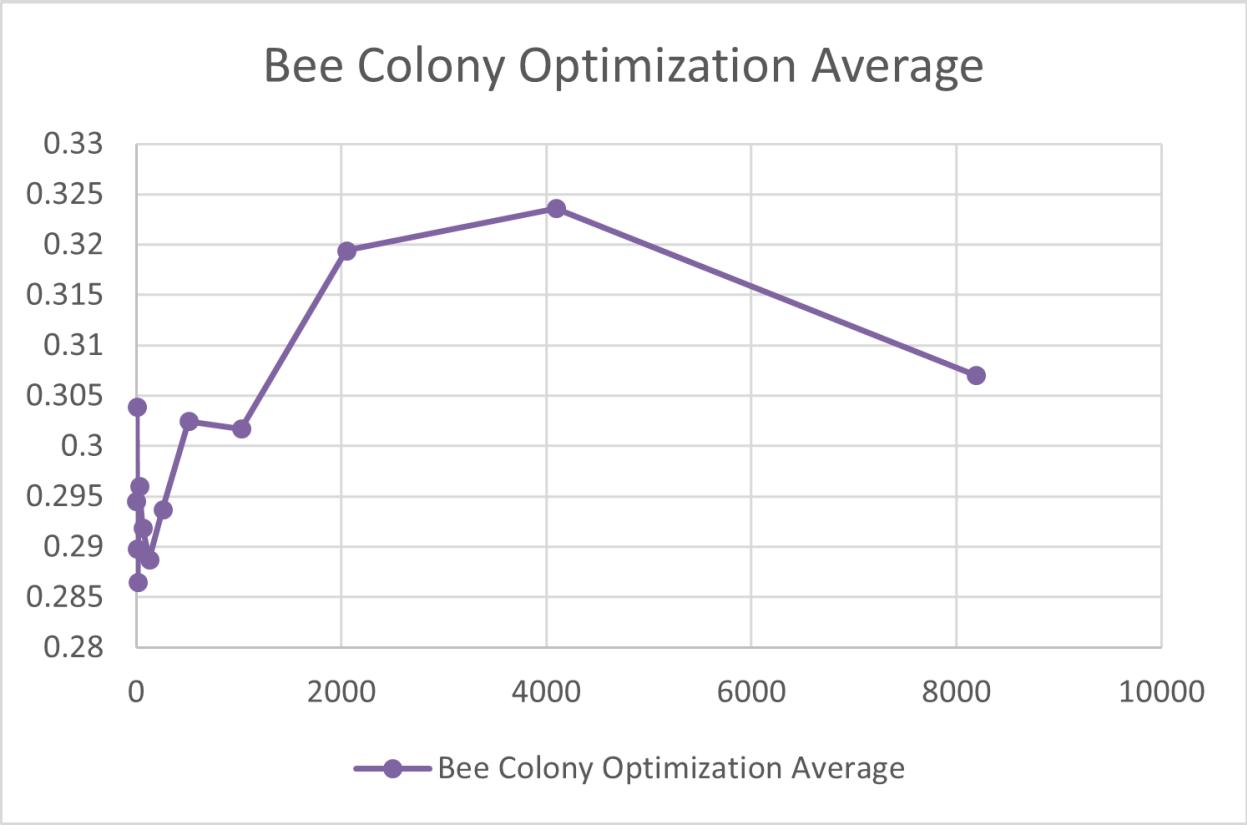
Results

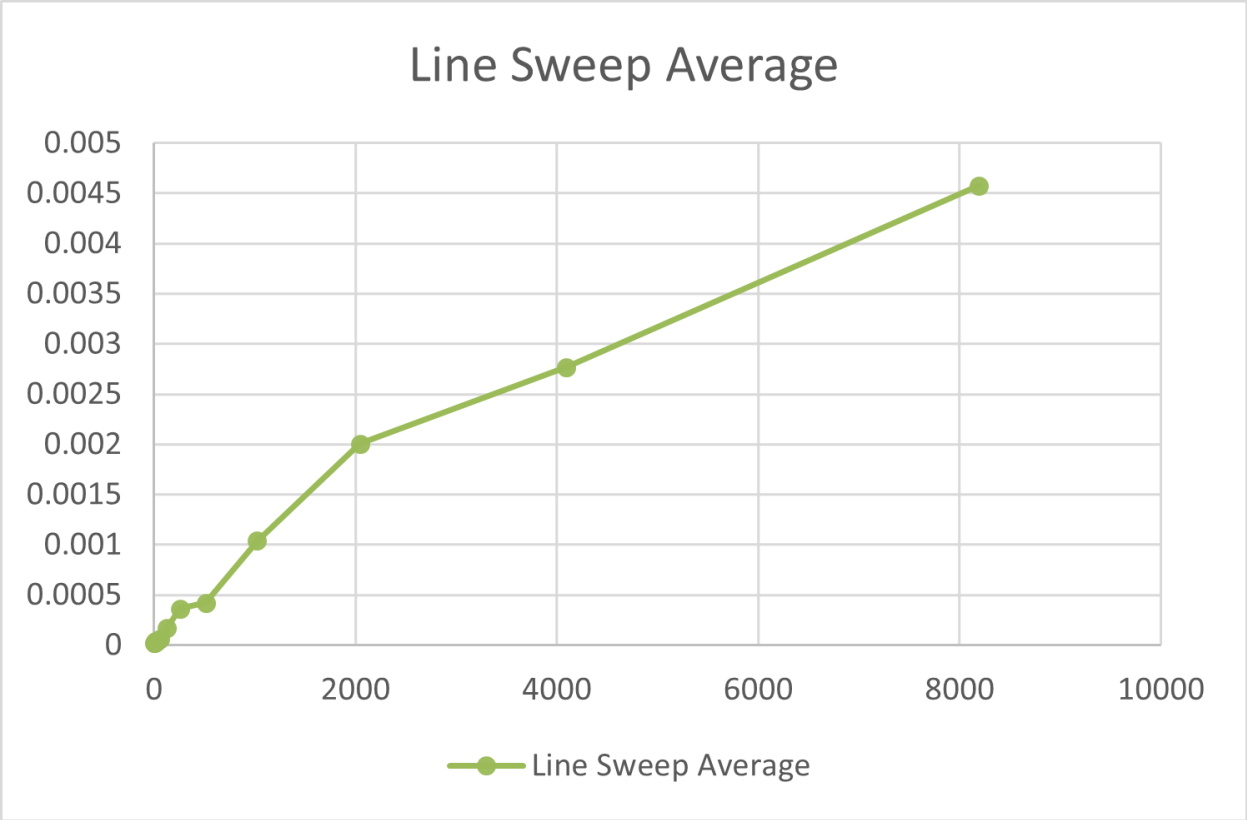



No Brute Force Average
0.35
0.3
0.25
0.2
0.15
0.1
0.05
0
0
1000
2000
3000
4000
5000
6000
7000
8000
9000
Divide and Conquer Average
Line Sweep Average
Bee Colony Optimization Average


Brute Force Average
16
14
12
10
8
6
4
2
0
0
2000
4000
6000
8000
10000
Brute Force Average


Divide and Conquer Average
0.05
0.045
0.04
0.035
0.03
0.025
0.02
0.015
0.01
0.005
0
0
2000
4000
6000
8000
10000
Divide and Conquer Average


Bee Colony Optimization Average
0.33
0.325
0.32
0.315
0.31
0.305
0.3
0.295
0.29
0.285
0.28
0
2000
4000
6000
8000
10000
Bee Colony Optimization Average



Dataset Size	Trial	Brute Force Times	Brute Force Average	Divide and Conquer Times	Divide and Conquer Average	Line Sweep Times	Line Sweep Average	Bee Colony Optimization Times	BCO Average	Actual Closest Distance	BCO Closest Distance	Average Difference
2	1	1.98E-05		2.08E-05		1.63E-05		0.301871		6547.424156	6547.424156	
2	2	1.54E-05		2.21E-05		1.97E-05		0.2885979		8643.224925	8643.224925	
2	3	1.46E-05		2.09E-05		2.02E-05		0.2962164		12464.9977	12464.9977	
2	4	1.56E-05		1.95E-05		1.74E-05		0.291973		1448.602627	1448.602627	
2	5	1.5E-05	1.608E-05	1.71E-05	2.008E-05	1.43E-05	1.758E-05	0.2941879	0.29456924	1681.877973	1681.877973	0
4	1	2.51E-05		3.65E-05		2.38E-05		0.2893279		1718.957156	1718.957156	
4	2	1.91E-05		4.51E-05		2.6E-05		0.3027446		3313.699347	3313.699347	
4	3	1.86E-05		3.94E-05		3.01E-05		0.2930155		2437.465499	2437.465499	
4	4	1.97E-05		3.7E-05		2.1E-05		0.2841041		1448.602627	1448.602627	
4	5	2.19E-05	2.088E-05	3.03E-05	3.766E-05	2.18E-05	2.454E-05	0.2796285	0.28976412	1681.877973	1681.877973	0
8	1	4.05E-05		0.0001044		2.94E-05		0.2957756		1718.957156	1718.957156	
8	2	3.72E-05		5.54E-05		2.61E-05		0.2772258		1917.26952	1917.26952	
8	3	4.13E-05		6.43E-05		3.05E-05		0.274965		1739.628053	1739.628053	
8	4	9.49E-05		5.87E-05		2.65E-05		0.336553		861.2815114	861.2815114	
8	5	4.04E-05	5.086E-05	5.18E-05	6.692E-05	6.08E-05	3.466E-05	0.3348778	0.30387944	634.5533534	634.5533534	0
16	1	0.0001156		6.44E-05		3.32E-05		0.2756306		165.0522479	165.0522479	
16	2	0.0001087		0.0001172		3.18E-05		0.2858901		738.7398326	738.7398326	
16	3	0.000163		9.35E-05		2.78E-05		0.2943701		540.7473974	540.7473974	
16	4	0.0001401		8.51E-05		2.98E-05		0.2842168		292.8838643	292.8838643	
16	5	0.0001147	0.00012842	0.0001256	9.716E-05	3.01E-05	3.054E-05	0.2923055	0.28648262	453.5276976	453.5276976	0
32	1	0.0004266		0.0001707		4.89E-05		0.2995403		165.0522479	165.0522479	
32	2	0.0003557		0.0001237		3E-05		0.3010687		217.5125125	217.5125125	
32	3	0.0003233		0.0002229		4.12E-05		0.2880597		275.7939447	275.7939447	
32	4	0.0003965		0.0002143		4.74E-05		0.284005		266.9508182	266.9508182	
32	5	0.0003843	0.00037728	0.0002387	0.00019406	7.91E-05	4.932E-05	0.3074357	0.29602188	259.8684557	259.8684557	0
64	1	0.0010198		0.0003092		7.1E-05		0.29949		117.2871557	117.2871557	
64	2	0.0010466		0.0004436		6.19E-05		0.281586		79.84815763	79.84815763	
64	3	0.0010097		0.0003526		7.25E-05		0.2935785		169.2146312	169.2146312	
64	4	0.000905		0.0002319		4.97E-05		0.2844557		27.28209519	27.28209519	
64	5	0.0014787	0.00109196	0.0004009	0.00034764	4.22E-05	5.946E-05	0.3002765	0.29187734	105.1175458	105.1175458	0
128	1	0.0035415		0.0009507		0.0001385		0.3042741		74.14092271	74.14092271	
128	2	0.0039988		0.0009988		0.0002811		0.2900915		52.87352978	52.87352978	
128	3	0.0034539		0.0010287		0.0001167		0.2808568		51.25611577	51.25611577	
128	4	0.0036395		0.0005192		0.0001772		0.2810693		27.28209519	27.28209519	
128	5	0.0034614	0.00361902	0.0011592	0.00093132	0.0001437	0.00017144	0.2875502	0.28876838	53.98433152	53.98433152	0
256	1	0.0138672		0.0010079		0.000324		0.2879555		5.251194271	73.76580682	
256	2	0.0156798		0.0015853		0.0002797		0.2871365		52.87352978	91.29493667	
256	3	0.0267987		0.0016506		0.0002171		0.293883		3.879427369	34.17506637	
256	4	0.0155414		0.0019939		0.0002513		0.2988836		20.9157177	33.59095937	
256	5	0.0135751	0.01709244	0.0021292	0.00167338	0.0007331	0.00036104	0.3008128	0.29373428	44.20631108	47.12246106	30.56461002
512	1	0.0648724		0.0021176		0.0004132		0.3219445		5.251194271	47.44273572	
512	2	0.0611288		0.0024085		0.000533		0.2881322		14.26785969	35.99614349	
512	3	0.0672348		0.0023659		0.0003715		0.3070496		3.879427369	30.15148751	
512	4	0.0756386		0.0020504		0.0003045		0.283998		11.96957943	48.0348978	
512	5	0.0654447	0.06686386	0.0021384	0.00221616	0.0004867	0.00042178	0.3112279	0.30247044	15.95549809	67.26872895	35.51408693
1024	1	0.248726		0.004325		0.0018213		0.3049015		5.251194271	90.11941073	
1024	2	0.2260859		0.0043734		0.0008745		0.3002195		6.93202623	50.73672808	
1024	3	0.2652347		0.004185		0.0010885		0.295943		3.879427369	72.14867287	
1024	4	0.2379605		0.00437		0.0007511		0.3053177		3.128848584	114.1622246	
1024	5	0.2532136	0.24624414	0.0045467	0.00436002	0.0006573	0.00103854	0.3022258	0.3017215	6.777462231	37.18532224	67.67667996
2048	1	0.9877237		0.0103759		0.0017953		0.3470073		1.915652249	15.63866383	
2048	2	0.9243247		0.0088625		0.0022087		0.3029395		3.337685173	53.48919027	
2048	3	0.9294924		0.0092945		0.0017597		0.3085305		3.879427369	13.12888694	
2048	4	0.9192362		0.0088911		0.0018386		0.3074633		1.503987452	46.9521655	
2048	5	0.9737775	0.9469109	0.0100256	0.00948992	0.002431	0.00200666	0.3310846	0.31940504	2.404152229	11.55223545	25.5440475
4096	1	3.7374444		0.0332927		0.0025975		0.3464981		1.915652249	75.62030857	
4096	2	3.8174957		0.0197259		0.0030851		0.3362473		2.097576238	83.88245602	
4096	3	3.928645		0.0210881		0.00297		0.2973143		1.73128631	33.92208025	
4096	4	3.9654939		0.0253823		0.002162		0.2961574		1.503987452	19.51637886	
4096	5	3.7891317	3.84764214	0.0242445	0.0247467	0.0030281	0.00276854	0.3417159	0.3235866	2.404152229	53.69614241	51.39694232
8192	1	14.9561561		0.0430424		0.0043657		0.2972508		0.352944242	38.23335891	
8192	2	14.9497449		0.0462594		0.0047608		0.2947542		0.963019192	27.95115497	
8192	3	15.25885		0.0495284		0.0043765		0.2984171		0.781419973	106.1199331	
8192	4	15.465246		0.0473958		0.0045906		0.3237815		0.638267742	52.20177634	
8192	5	15.2956821	15.18513582	0.0494324	0.04713168	0.004781	0.00457492	0.320838	0.30700832	0.925137407	72.4298604	58.65505904

Interpreting Results

Looking at the experimental results, we can conclude that both research questions were addressed through empirical analysis of the four algorithms, focusing on runtime behavior and performance crossover points.

The runtime results show that the Brute Force algorithm scales quadratically with input size, confirming its $\Theta(n^2)$ time complexity. In contrast, both the Divide and Conquer and Line Sweep algorithms exhibit $\Theta(n \log n)$ growth, resulting in significantly better performance as n increases. Bee Colony Optimization (BCO) showed less predictable scaling but showed a time complexity of $\Theta(N \cdot D \cdot I)$. However, unlike the other algorithms, BCO does not guarantee an exact solution, which introduces a trade-off between runtime efficiency and accuracy. In the end, the algorithm with the fastest runtime was Line Sweep, and the slowest was the Brute Force algorithm.

The results show that the Divide and Conquer algorithm begins to outperform the Brute Force approach as the input size increases. For small input sizes, the overhead of recursion and additional processing causes Divide and Conquer to perform similarly to, or slightly worse than, Brute Force. However, as n increases, its superior $\Theta(n \log n)$ time complexity leads to significantly better performance. This trend is evident in the experiment results. At $n = 2$, Brute Force required approximately 1.60×10^{-5} seconds, while Divide and Conquer required approximately 2.0×10^{-5} seconds, showing nearly identical performance but Brute Force slightly faster. n = 16 is where Divide and Conquer first starts to beat out Brute Force, where Divide and Conquer had a time of 9.72×10^{-5} seconds and Brute Force had a time of 1.28×10^{-4} seconds. At $n = 8192$, Brute Force required approximately 15.19 seconds, whereas Divide and Conquer required only about 0.047 seconds to compute the closest pair. This dramatic difference highlights the impact of asymptotic complexity on performance at large input sizes.

Reflection

We had to make a couple decisions regarding our project from our original plan. One of those decisions was to generate a dataset instead of using a real dataset. This came from the fact that we had misinterpreted the data on the website that we originally picked, and when looking for another real dataset, found that there really were none that would be consistent and were the size that we needed. Synthetically generating random XY pairs proved way more efficient and time saving.

Another portion of the project that came up was how heuristic BCO was. When comparing its result to the closest pair problem with the others, it had varying levels of accuracy. This accuracy changed depending on the number of bees and iterations we used in the function. At first, we thought about adding a section to our results to test different values of bees and iterations, but we ultimately decided that that wasn't what we wanted to prove in the project. We wanted to focus on comparing runtimes across the 4 algorithms and not the effects the number of bees and iterations would have on accuracy and runtime for BCO.

Conclusion

Through this project, we wanted to figure out how the runtime of each algorithm scales with increasing input size n, and at what input size Divide & Conquer outperforms Brute Force. We found through our experiments that Brute Force scales exponentially, BCO's runtime is constant based on the number of bees and iterations it has, and Line Sweep takes the smallest amount of time, out of the 4 algorithms that we tested, to solve the Closest Pair Problem. We also concluded that Brute Force is better than Divide and Conquer at input size $n < 16$.

Participation

Objectives	Lydia Ballard	Sydney Frisbee
Planning	50%	50%
Coding	66%	33%
Result Generation	66%	33%
Report Writing	33%	66%
Presentation	50%	50%
Communication	50%	50%

Sources

"Closest Pair of Points." *GeeksforGeeks*, 21 Apr. 2026, www.geeksforgeeks.org/dsa/closest-pair-of-points-using-divide-and-conquer-algorithm/.

"ChatGPT." OpenAI, 2025, chat.openai.com/.

- The author(s) acknowledge the utilization of ChatGPT, a language model developed by OpenAI, in the preparation of this assignment. ChatGPT was employed in the following manner(s) within this assignment: code generation in sections of our 4 python algorithm files.